

Author :Jay Jiao

1_1

编写程序计算三角函数 $\sin x$ 近似值。请根据 $\sin x$ 泰勒展开式，应用循环编写程序。

$$\sin x = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)!} x^{2k+1} = x - \frac{1}{3!}x^3 + \frac{1}{5!}x^5 - \frac{1}{7!}x^7 + \dots$$

当 $\left| \frac{(-1)^k}{(2k+1)!} x^{2k+1} \right| < 10^{-9}$ 时，循环结束。

```
#include <iostream>
#include<cmath>
using namespace std;
int main()
{
    double x, sinX = 0;
    cout << "Please input the value of x ): ";
    cin >> x;
    //请补充完成以下缺少的代码
    double item=x;
    for(int i=1;abs(item)>=0.000000001;i++){
        sinX+=item;
        item=item*(-1)*x*x/(2*i+1)/(2*i);
    }

    cout << "sin(" << x << ") = " << sinX << endl;
    return 0;
}
```

1_2

请编写程序通过循环控制：计算并打输出 1 到 n（包含 n）之间所有质数的和，其中 n 是用户输入的正整数。

```
#include <iostream>
using namespace std;
int main() {
    int n;
    int sum = 0;
    //向用户请求输入
    cout << "Please enter a positive integer:";
    cin >> n;
    //编写一个循环来计算质数之和
```

```

//请在这里编写代码
    if(n>=2) sum+=2;
    for(int i=3;i<=n;i+=2)
    {
        int count=0;
        for(int j=3;j<=i;j++)
        {
            if(i%j==0) count++;
        }
        if(count==1) sum+=i;
    }
//输出结果
cout << "The sum of primes:" << sum << endl;
return 0;
}

// 编写一个循环来计算质数之和
for (int i = 2; i <= n; i++) {
    bool isPrime = true;
    for (int j = 2; j * j <= i; j++) {
        if (i % j == 0) {
            isPrime = false;
            break;
        }
    }
    if (isPrime) {
        sum += i;
    }
}
// 打印结果

```

1_3

根据级数展开，当 x 的取值范围为 $-1 < x < 1$ 时，

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \dots (-1)^{n-1} \frac{x^n}{n} + \dots$$

请编写循环程序计算 $\ln(1+x)$ ，并输出计算结果，注意：（1）请设置循环的结束条件为新增的项的绝对值小于 0.0000001；（2）你可根据需要定义新的变量

```

#include <iostream>
using namespace std;
int main() {
    double x;
    double result = 0;
    //向用户请求输入
    do {
        cout << "Please enter a decimal number x (-1 < x < 1): ";
    }
}

```

```

        cin >> x;
    } while (x <= -1 || x >= 1);
    //请编写代码计算 ln(1+x), 结果存储在 result 中
    //请在这里补充缺少的代码
    double item=x;
    for(int i=1;item>=0.0000001 || item<=-0.0000001;i++)
    {
        result+=item;
        item=item*(-1)*x*i/(i+1);
    }
    //输出结果
    if (x >= 0) cout << "ln(1+" << x << ")=" << result << endl;
    else cout << "ln(1" << x << ")=" << result << endl;
    return 0;
}

```

2_1

编写名为 calculateStatistics 的函数，计算整型数组元素中的平均值、方差，以及数组中大于等于平均值的元素个数和小于平均值的元素个数。

数组元素平均值：

$$x_mean = \sum_{i=1}^N x_i$$

数组元素的方差：

$$x_deviation = \frac{1}{N} \sum_{i=1}^N (x_i - x_mean)^2$$

```

#include <iostream>
using namespace std;
const int NUM = 10;
double calculateStatistics(int *x,int NUM,double *x_deviation,int &greaterNum, int&lessNum);
int main()
{
    int x[NUM] = {12, 23, 41, 28, 19, 27, 38, 14, 52, 32};
    int greaterNum = 0, lessNum = 0;
    double x_mean = 0, x_deviation = 0;
    x_mean = calculateStatistics(x, NUM, &x_deviation, greaterNum, lessNum);
    cout << "数组元素平均值: " << x_mean << endl
        << "数组元素的方差: " << x_deviation << endl
        << "大于等于平均值的元素: " << greaterNum << "个" << endl
        << "小于平均值元素: " << lessNum << "个" << endl;
    return 0;
}
//请编写 calculateStatistics 的函数

```

```
double calculateStatistics(int *x,int NUM,double *x_deviation,int &greaterNum, int&lessNum)
{
    double mean;
    for(int i=0;i<NUM;i++)
    {
        mean+=x[i];
    }
    mean/=NUM;
    for(int j=0;j<NUM;j++)
    {
        if(x[j]>=mean) greaterNum++;
        else lessNum++;
        *x_deviation+=(x[j]-mean)*(x[j]-mean);
    }
    *x_deviation/=NUM;
    return mean;
}
```

2_2 请编写一个函数模板，该模板接受一个数组（通过指针传递）和一个整数 *k*，并返回数组中第 *k* 大的元素。主函数应该能够处理任何类型的数据。提示：为了找出第 *k* 大的元素，可能需要先对数组进行排序。

```
#include <iostream>
using namespace std;
//请在这里定义函数模板
template <class T> T kthLargest(T *a,int n,int k)
{
    for(int i=1;i<n;++i)
    {
        bool flag =false;
        for(int j=0;j<n-i;++j)
        if(a[j+1]<a[j]){
            T tmp =a[j];
            a[j]=a[j+1];
            a[j+1]=tmp;
            flag = true;
        }
        if(!flag) break;
    }
    return a[n-k];
}

int main() {
    int arr[] = {5, 10, 15, 20, 25};
```

```

int n = sizeof(arr) / sizeof(arr[0]);
int k = 2;
//在这里调用函数模板来找到数组中第 k 大的元素
int kth = kthLargest(arr, n, k);
cout << "The " << k << "th largest element is " << kth << endl;
return 0;
}

```

3_1 本题目的程序可以对输入的 3 个句子(包含空格的字符串)进行升序排序。并输出排好序的句子。

```

#include <iostream>
#include <cstring>
using namespace std;

const int MAXNUM = 3;
const int MAXLENGHT = 80;
void stringSort(char str[][MAXLENGHT], int n) {
    char *temp = new char[MAXLENGHT];
    for (int i = 0; i < MAXNUM - 1; i++) {
        int k = i;
        for (int j = i + 1; j < MAXNUM; j++)
            if (strcmp(str[k], str[j]) > 0) k = j;
        if (k != i) {
            strcpy(temp, str[i]);
            strcpy(str[i], str[k]);
            strcpy(str[k], temp);
        }
    }
}

int main() {
    char sentences[MAXNUM][MAXLENGHT];
    cout << "Please input three sentences:" << endl;
    for (int i = 0; i < MAXNUM; i++)
        cin.getline(sentences[i], MAXLENGHT);
    cout << endl;
    stringSort(sentences, MAXNUM);
    cout << "Sorted sentences: " << endl;
    for (int i = 0; i < MAXNUM; i++)
        cout << sentences[i] << endl;
    return 0;
}

```

3_2 现在有 n 个长度是 k 的字符串，字符串中的字符都是由小写字母和大写字母组成。现在想把这 n 个字符串按字典混乱序从小到大排序。字典 混乱序：我们首先定义字母

上的大小顺序： $a < b < c < \dots < z < Z < Y < X < \dots < B < A$ 。

那么当 S1 和 S2 是两个字符串：

- 1) $S1=S2$ 当且仅当 S1 和 S2 完全一样;
- 2) $S1<S2$ 当且仅当 S1 和 S2 的第一个不同的字符是位置 i, 并且如果 $S1[i] < S2[i]$;
- 3) $S1>S2$ 当且仅当 S1 和 S2 的第一个不同的字符是位置 i, 并且 $S1[i] > S2[i]$.

```
#include <iostream>
#include <string>
using namespace std;
bool isBigger(string s1, string s2, int k) {
    //请在此处完善修改此函数代码
    for(int i=0;i<k;i++)
    {
        if(s1[i]==s2[i]) continue;
        if(s1[i]<'Z'&& s1[i]>'A'){
            if(s2[i]<'z'&& s2[i]>'a') return true;
            else return (s1[i]<s2[i]);
        }
        else{
            if(s2[i]<'z'&& s2[i]>'a') return (s1[i]>s2[i]);
            else return false;
        }
    }
    return true;
}
int main() {
    int n, k;
    cin >> n >> k;
    string st[200];
    for (int i = 0; i < n; i++) cin >> st[i];
    //排序部分(sorting)
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (isBigger(st[i], st[j], k)) {
                string tmp;
                tmp = st[i];
                st[i] = st[j];
                st[j] = tmp;
            }
        }
    }
    //输出排好序的字符串数组
```

```

    for (int i = 0; i < n; i++) cout << st[i] << endl;
    return 0;
}

```

3_3 请编写一个函数，比较两个字符串的大小。该函数原型为 `int strcmp(char *s1, char *s2)`，字符串 `s1`、`s2` 均为英文字母、数字等组成的字符串，请按照字典方式及 Ascii 序进行比较，字符串 `s1` 大于/等于/小于字符串 `s2` 时，分别输出 1/0/-1。

```

#include <iostream>
using namespace std;
int mystrcmp(char *s1, char *s2);
int main() {
    cout << "以下代码用于测试 mystrcmp 函数" << endl;
    cout << "strcmp(\"hello\", \"heaven\")=" << strcmp("hello", "heaven") << endl;
    cout << "strcmp(\"world better\", \"worse\")=" << strcmp("world better", "worse") <<
endl;
    cout << "strcmp(\"hello 123!\", \"hello 123!\")=" << strcmp("hello 123!", "hello 123!")
<< endl;
    cout << "strcmp(\"hollywood\", \"holly\")=" << strcmp("hollywood", "holly") << endl;
    cout << "strcmp(\"holly\", \"hollywood\")=" << strcmp("holly", "hollywood") << endl;
    return 0;
}
//请编写代码实现如下函数
int mystrcmp(char *s1, char *s2) {
    int i;
    for(i=0; s1[i]!='\0'; i++)
    {
        if(s1[i]==s2[i]) continue;
        if(s1[i]>s2[i]) return 1;
        else return -1;
    }
    if(s2[i]!='\0') return -1;
    return 0;
}

```

4_11 1) 将 3 个字符串，每个字符串去除其中非数字和非英文字母的字符，并将小写字母转化为大写字母，然后进行连接合并；

2) 统计 3 个字符串中数字字符和英文字母的频次；

3) 输出连接合并后的字符串、以及 3 个字符串中数字字符和英文字母的频次。

例如：`const char *str[3] = { "I love VC++ 2010!",
"Do you like Python 3.7?",`

```
"Java SE18 is good!"
```

```
};
```

去除非英文字母和非数字的字符 ↓ 小写字母转化为大写字母, 进行连接合并

```
mergestring: ILOVEVC2010DOYOU LIKEPYTHON37JAVASE18ISGOOD
```

中英文字母和数字字符的频次: alphabet[], digital[]

输出连接合并串 ↓ 英文字母和数字字符的频次

```
Merged strings: ILOVEVC2010DOYOU LIKEPYTHON37JAVASE18ISGOOD
```

数字字符的频次: 0:2 1:2 2:1 3:1 7:1 8:1

英文字母的频次: A:2 C:1 D:2 E:3 G:1 H:1 I:3 J:1 K:1 L:2 N:1 O:6 P:1 S:2 T:1 U:1 V:3 Y:2

仔细阅读实现上述功能的 MergeStatistics 函数的实现, 找出其中的错误, 并 (在不改变程序结构基础上) 进行修改。

提示: 注意用于计算频次的数组下标、大小写转换、内存空间的动态分配等。

```
#include <iostream>
#include <cstring>
#include <cctype>
using namespace std;
const int MAXLENGTH = 3;
//去除非英文字母和非数字的字符,小写字母转化为大写字母,返回连接合并的字符串指针
char *MergeStatistics(const char *str[], int MaxLen, int digit[], int alphabet[]) {
    int mergedlen = 0;
    for (int i = 0; i < MaxLen; i++)
        mergedlen += strlen(str[i]);
    char *mergestring = new char[mergedlen + 1];
    int counter = 0;
    for (int i = 0; i < MaxLen; i++) {
        const char *ptr = str[i];
        char c;
        while ((c = *ptr) != '\0') {
            if (c >= 'a' && c <= 'z')    c = c - 32;
            if(c>='A'&&c<='Z') {
                int index = c-'A';
                alphabet[index]++;
                mergestring[counter++] = c;
            }
            if (c >= '0' && c <= '9') {
                int index = c - '0';
                digit[index]++;
                mergestring[counter++] = c;
            }
            ptr++;
        }
    }
}
```



```

        mergestring[counter] = '\0';//别忘了结束符
        return mergestring;
    }
//以下主程序无须修改
int main() {
    const char *str[MAXLENGTH] = {"I love VC++ 2010!",
                                    "Do you like Python 3.7?",
                                    "Java SE18 is good"};

    int digital[10] = {0};//存放数字字符频次的数组,
    int alphabet[26] = {0};//存放英文字母频次的数组
    //去除非英文字母和非数字的字符,小写字母转化为大写字母,返回连接合并的字符串指针
    char *mergedStr = MergeStatistics(str, MAXLENGTH, digital, alphabet);
    cout << "Merged strings:" << mergedStr << endl;//输出连接合并的字符串
    for (int i = 0; i < 10; i++)//输出数字字符的频次
        if (digital[i]) cout << i << ": " << digital[i] << " ";
    cout << endl;
    for (int i = 0; i < 26; i++)//输出英文字母的频次
        if (alphabet[i]) cout << char(i + 'A') << ": " << alphabet[i] << " ";
    delete[] mergedStr;//释放 new 申请的空间
    return 0;
}

```

4_2 对于本题中的 `remove_repeat` 函数，该函数输入一个有序数组，通过指针返回一个去重后的数组，同时通过引用参数传递的方式返回去重后数组的大小，以指针参数传递的方式返回重复元素的最小和最大重复次数（数据保证有重复元素）。

请在**保持函数参数列表**和**返回值类型不能变**的前提下，修正下面 `main` 函数和 `remove_repeat` 函数中不正确的代码。

```

#include <iostream>
#include<cmath>
using namespace std;
int* remove_repeat(int a[], int n, int &m, int *minr, int *maxr) { //不允许改变此行
    int *b = new int[n];
    int r = 1;
    b[m++] = a[0];
    for (int i = 1; i < n; i++) {
        if (b[m - 1] != a[i]) {
            b[m++] = a[i];
            if(r>1){
                *minr = min(*minr, r);
                *maxr = max(*maxr, r);
            }
        }
    }
}

```

```

        r = 1;
    } else {
        r++;
    }
}
return b;
}

int main() {
    int a[] = {1, 2, 2, 2, 2, 3, 3, 3, 4, 5};
    int n = sizeof(a) / sizeof(int); // which is equal to the number of elements in a[]
    int m=0, minr=n, maxr=0;
    int *p = remove_repeat(a, n, m, &minr, &maxr);
    cout << m << endl; // should output: 5
    for (int i = 0; i < m; i++)
        cout << p[i] << ' '; // should output: 1 2 3 4 5
    cout << endl;
    cout << minr << ' ' << maxr << endl; // should output: 3 4

    delete[] p;
    return 0;
}

```

4_3 函数 `char * addLongInt(char *x1, char *x2)` 将两个正整数 x1, x2 相加，返回相加后的结果返回结果的指针指向一个动态字符数组。注意 x1, x2 均为字符串，请补充完成该函数请实现该函数。

```

#include <iostream>
#include <cstring>

using namespace std;
char *addLongInt(char *x1, char *x2);
int main() {
    char *result;
    result = addLongInt("220", "7890");
    cout << "220 + 7890 = " << result << endl;
    result = addLongInt("220020123456789", "1234567890123456789");
    cout << "220020123456789 + 1234567890123456789 = " << result << endl;
    return 0;
}

char *addLongInt(char *x1, char *x2) {

```

```

int len1 = strlen(x1);
int len2 = strlen(x2);
int len = len1 > len2 ? len1 : len2; //计算出两个整数中最长整数的位数
int curr = 0; //用于记录当前位的计算结果
int carry = 0; //用于记录是否有进位
//下面语句不完整，请补充完成以下语句，让 sum 指向一个动态字符数组
char *sum = new char [len+2];

//请补充完成以下代码。这段代码依次将 x1 与 x2 的对应位相加，判断是否有进位，
并将计算结果存储在 sum 指向的动态数组中。为计算方便，请先按照反向顺序存储结果，
例如：如果相加的和为“8110”，则按照“0118”存储在动态数组中。请注意(1)处理进位，(2)
两个相加的数的长度可能不同。

//下面代码将存储在动态数组中字符串顺序变成反向的，例如：将"0118"变为"8110"
int i = len1 - 1, j = len2 - 1, k = 0;
while (i >= 0 || j >= 0 || carry > 0) {
    int digit1 = (i >= 0) ? x1[i] - '0' : 0;
    int digit2 = (j >= 0) ? x2[j] - '0' : 0;
    int total = digit1 + digit2 + carry;
    sum[k++] = (total % 10) + '0'; // 存储当前位的结果
    carry = total / 10; // 更新进位
    i--;
    j--;
}
len = k;
for (int i = 0; i < len / 2; i++) { //注：这里的 len 记录的是求和结果的最后长度
    char temp;
    temp = sum[i];
    sum[i] = sum[len - i - 1];
    sum[len - i - 1] = temp;
}
sum[len] = '\0'; //加字符串结束标记
return sum;
}

```

5_1 两个非负整数的最大公约数（经常被简写为 gcd）就是能整除这两个整数的因数中最大的。

希腊数学家欧几里得发现两个非负整数 x, y 的最大公约数可以用如下的方法计算：

- (I) 如果 x 可以被 y 整除，则 y 就是最大公约数；
- (II) 否则， x 和 y 的最大公约数总是等于 y 和 y 除 x 所得的余数的最大公约数。也即：
 $\text{gcd}(x, y) == \text{gcd}(y, y \text{ 除 } x \text{ 所得的余数})$

```

#include<iostream>
using namespace std;

```

```

int gcd(int x,int y)
{
    if(y==0) return x;
    if(x%y==0) return y;
    else
    {
        return gcd(y,x%y);
    }
}
int main()
{
    int x,y;
    cin>>x>>y;
    cout<<gcd(x,y)<<endl;
    return 0;
}

```

/*y 除 x 的余数 x%y 总是小于 y, 因此函数的第二个参数总是递减, 因此总有一次调用时 y 整除 x (或是 y 递减过程中, 或是当 y 变为 1 时), 从而保证递归结束。*/

5_2 现在有一排 n 个鸡窝: 1,2,...,n。第 i 个鸡窝里面有 num[i]个鸡蛋。现在你想从这些鸡窝里面拿出来尽量多的鸡蛋。但是问题是如果你从相邻两个鸡窝里面拿鸡蛋的话, 母鸡会非常生气, 会把你啄伤。问题: 如何在不被啄伤的情况下拿出来尽量多鸡蛋。请完善下面的**递归函数**。

```

#include <iostream>
using namespace std;
int value(int num[], int n) {
//请在此处完善修改此递归函数代码
    if (n==1) return num[1];
    if(n==2) return (num[1]>num[2])? num[1]:num[2];
    int choice1=value(num,n-1);
    int choice2=value(num,n-2)+num[n];
    return choice1>choice2? choice1:choice2;
}
int main() {
    int num[100] = {0};
    int n;
    cin >> n;
    for (int i = 0; i < n; i++) cin >> num[i];
    cout << value(num, n) << endl;
}

```

```

    return 0;
}

```

5_3 请将英文文本进行如下变换，a 变换为 z，b 变换为 y，c 变换为 z，..., m 变换为 n, ..., z 变换为 a，你可用如下方式 $ch = ch + 26 - 2*(ch - 'a') - 1$ ；该英语文本中字符仅包括 26 个小写英文字母以及空格。
要求：请写一个递归函数实现，将变换后的结果输出到屏幕。

```

#include <iostream>
#include <cstring>

using namespace std;
void transformtext(const char *s, int n);
//注：上面函数输出字符串 s 的第 n 个字符（包括第 n 个字符）之后的子串
int main() {
    transformtext("abcdefghijklmnopqrstuvwxyz", 0);
    cout << endl;
    transformtext("hello world", 0);
    cout << endl;
    transformtext("svooll diow", 0);
    return 0;
}
void transformtext(const char *s, int n)
{
    if(s[n] == '\0') return;
    if(s[n] == ' ') cout << ' ';
    if(s[n] <= 'z' && s[n] >= 'a')
    {
        char t = s[n] + 26 - 2*(s[n] - 'a') - 1;
        cout << t;
    }
    transformtext(s, n+1);
}

```

6_1

//BurgerShop.cpp（类 BurgerShop 的实现文件）
#include <stdlib.h>
#include <cstring>
#include <cmath>

```

#include <iostream>
#include "BurgerShop.h"
using namespace std;
BurgerShop::BurgerShop() {
    burger_arr = NULL;
    burger_num = 0;
}
BurgerShop::BurgerShop(const BurgerShop & burgerShop) {
    //请补充缺少以下的代码 6.1
    burger_num=burgerShop.burger_num;
    burger_arr = new Burger[burger_num];
    for(int i=0;i<burger_num;i++)
    {
        burger_arr[i].price = burgerShop.burger_arr[i].price;
        burger_arr[i].nutrient = burgerShop.burger_arr[i].nutrient;

    }
}
void BurgerShop::AddBurger(Burger burger) {
    //请补充缺少以下的代码 6.2

    if(burger_num%ALLOC_SPACE==0)
    {
        Burger* temp = new Burger[burger_num+ALLOC_SPACE];
        for(int i=0;i<burger_num;i++)
            temp[i]=burger_arr[i];
        if(burger_arr!=NULL) delete[] burger_arr;//注意这一步
        burger_arr=temp;
    }
    burger_arr[burger_num]=burger;

    burger_num++;
}
// 注：调用 cout << burgerShopA; 在屏幕上的输出的示例如下，请据此来编写 operator<<
重载函数。
ostream & operator<<(ostream & os, const BurgerShop bp) {
    //请补充缺少以下的代码 6.3
    os<<"burger number: "<<bp.burger_num<<endl;
    for(int i=0;i<bp.burger_num;i++)
        os<<"burger      "<<i+1<<"->      price:      "<<bp.burger_arr[i].price<<"      nutrient:
"<<bp.burger_arr[i].nutrient<<endl;
    return os;
}

```

```
BurgerShop& BurgerShop::operator=(const BurgerShop &burgerShop) {
```

```
    //请补充缺少以下的代码 6.4
    if(&burgerShop==this) return *this;
    delete [] burger_arr;
    burger_num=burgerShop.burger_num;
    burger_arr = new Burger[burger_num];
    for(int i=0;i<burger_num;i++)
    {
        burger_arr[i].price = burgerShop.burger_arr[i].price;
        burger_arr[i].nutrient = burgerShop.burger_arr[i].nutrient;
    }
    return *this;
}
```

```
void BurgerShop::SellBurger(int index) {
    if (index > burger_num || index < 1) {
        cout << "no such burger\n";
        return;
    }
    for (int i = index - 1; i < burger_num - 1; i++)
        burger_arr[i] = burger_arr[i + 1];
    burger_num--;
}

Burger & BurgerShop::operator[ ](int index) {
    return burger_arr[index];
}

BurgerShop::~BurgerShop() {
    if (burger_arr!= NULL)
        delete[] burger_arr;
}
```

6_2

// 请补充以下类 intArray 的实现程序中缺少的代码

// 6.cpp

```
#include "intarray.h"
```

```
#include <iostream>
```

```
using namespace std;
```

```
//构造函数
```

```
IntArray::IntArray(int *arr, int len) {
```

```

    if (len!= 0) {
        content = new int[len];
        for (int i = 0; i < len; i++) content[i] = arr[i];
    } else content = nullptr; //如数组中没有元素,此时无需申请内存空间
    length = len;
}

//拷贝(复制)构造函数
IntArray::IntArray(const IntArray & other) {
    if (this == &other) return;
    //请补充以下缺少的代码
    content = new int [other.length];
    for(int i=0;i<other.length;i++) content[i]=other.content[i];
    length=other.length;
}

//析构函数
IntArray::~IntArray() {
    if (length!= 0) delete[] content;
}

//赋值运算符重载
IntArray & IntArray::operator=(const IntArray & right) {
    //请补充以下缺少的代码
    if (this == &right) return *this;
    //请补充以下缺少的代码
    content = new int [right.length];
    for(int i=0;i<right.length;i++) content[i]=right.content[i];
    length=right.length;
    return *this;
}

//重载运算符<<,用以显示字符串,要求所有元素在一行中显示,中间用逗号隔开,两边用大括号,
//示例: {237,-45,820}
ostream& operator<<(ostream & os, const IntArray& arr) {
    if (arr.length == 0) {
        os << "This is an empty array: {}";
        return os;
    }

    //请补充以下缺少的代码
    os<< "{";
    for(int i=0;i<arr.length-1;i++) os<<arr.content[i]<<",";
    os<<arr.content[arr.length-1]<<"}";
    return os;
}

//求数组的长度,即数组中元素的个数。

```



```

int IntArray::getLength() {
    //请补充以下缺少的代码
    return length;
}
//在数组中查找一个整数,找到该整数第一个出现的位置的下标,如果不存在返回-1
int IntArray::findElem(int x) {
    int pos = -1;
    //请补充以下缺少的代码
    for(int i=0;i<length;i++)
    {
        if(x==content[i]){
            pos = i;
            break;
        }
    }
    return pos;
}
//在数组中添加一个整数,放在数组的末尾
void IntArray::add(int x) {
    int* temp = new int[length + 1];
    for (int i = 0; i < length; i++) temp[i] = content[i];
    temp[length] = x;
    //请补充以下缺少的代码
    if(content!=NULL) delete[]content;
    content = temp;
    length++;
}

```

6_3

```

#include <iostream>
#include<string>
using namespace std;

```

//请完成 BankAccount 类的实现

```

class BankAccount {
public:
    BankAccount(string accountNumber, double balance) {
        //构造函数
        this->accountNumber=accountNumber;
        this->balance=balance;
        transactionCount=0;
        for (int i = 0; i < 100; i++) {

```

```

        transactionHistory[i] = "";
    }

}

void deposit(double amount) {
    //存款函数
    balance+=amount;
    addTransaction("存款",amount);

}

bool withdraw(double amount) {
    //取款函数
    if(balance<amount) return false;
    balance-=amount;
    addTransaction("取款",amount);
    return true;
}

bool transfer(BankAccount& other, double amount) {
    //转账函数
    if(&other==this) return false;
    if(balance<amount) return false;
    balance-=amount;
    addTransaction("转账",amount);

    other.balance+=amount;
    other.addTransaction("转入",amount);

    return true;
}

void printTransactionHistory() {
    //打印交易历史函数
    for(int i=0;i<transactionCount;i++)
        cout<<transactionHistory[i]<<endl;
}

private:
    string accountNumber;
    string transactionHistory[100]; //交易历史
    double balance;
    int transactionCount; //交易计数器
    void addTransaction(string type, double amount) {
        string transaction = type + ":" + to_string(amount) + ",Balance:" + to_string(balance);
    }
}

```

```

        if (transactionCount < 100) {
            transactionHistory[transactionCount] = transaction;
            transactionCount++;
        } else {
            for (int i = 0; i < 99; i++) {
                transactionHistory[i] = transactionHistory[i + 1];
            }
            transactionHistory[99] = transaction;
        }
    }
};

```

//以下为用于测试的主函数

```

int main() {
    string str = "ID12345";
    BankAccount acc(str, 3456.12);
    if (acc.withdraw(1000)) cout << "withdraw succeed!" << endl;
    else cout << "withdraw fail, money not enough." << endl;
    acc.deposit(200);
    BankAccount acc2("ID12190", 0);
    acc.transfer(acc2, 500);
    acc.printTransactionHistory();
    return 0;
}

```

ASCII 值	控制字符	ASCII 值	控制字符	ASCII 值	控制字符	ASCII 值	控制字符
0	NUL	32	(space)	64	@	96	`
1	SOH	33	!	65	A	97	a
2	STX	34	"	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	\$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	,	71	G	103	g
8	BS	40	(	72	H	104	h
9	HT	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DC1	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	X	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	TB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL

类与对象的两个重磅例题：

Rational

```

#ifndef 课后习题_MYRATIONAL_H
#define 课后习题_MYRATIONAL_H
#include <iostream>
using namespace std;

class Rational
{
private:

```

```

    int num;
    int den;
    void ReductFraction() ;

public:
    Rational(int n=0,int d=1){num=n;den=d;ReductFraction();}
    friend Rational operator+(const Rational &r1,const Rational &r2) ;
    friend Rational operator*(const Rational &r1,const Rational &r2) ;
    friend bool operator < (const Rational &r1,const Rational &r2) ;
    friend bool operator==(const Rational &r1,const Rational &r2) ;
    friend ostream & operator<<(ostream &out,const Rational &r);
    friend istream &operator>>(istream &in,Rational &r);
    Rational & operator++();
    Rational operator++(int);
    Rational &operator--();
    Rational operator--(int);
    void display() const{cout<<num<<"/"<<den;}
    explicit operator double()const
    {
        return( double(num)/den);
    }
};

#endif //课后习题_MYRATIONAL_H

#include <iostream>
using namespace std;

#include "myRational.h"
Rational operator+(const Rational &r1,const Rational &r2) {
    Rational temp;
    temp.num=r2.num*r1.den+r2.den*r1.num;
    temp.den=r2.den*r1.den;
    temp.ReductFraction();
    return temp;
}
Rational operator*(const Rational &r1,const Rational &r2) {
    Rational temp;
    temp.num=r2.num*r1.num;
    temp.den=r2.den*r1.den;
    temp.ReductFraction();
    return temp;
}

```

```

bool operator<(const Rational &r1,const Rational &r2) {
    return r2.num*r1.den<r1.num*r2.den;
}
bool operator==(const Rational &r1,const Rational &r2) {
    return r2.num==r1.num&& r2.den==r1.den;
}
void Rational::ReductFraction() {
    int tmp=(num>den)?den:num;
    for(;tmp>1;--tmp)

        if(num%tmp==0&&den%tmp==0){
            num=num/tmp;
            den=den/tmp;
            break;
        }

}
Rational & Rational::operator++()
{
    num+=den;
    return *this;
}
Rational Rational::operator++(int x)
{
    Rational tmp = *this;
    num+=den;
    return tmp;
}
Rational& Rational::operator--()
{
    num-=den;
    return *this;
}
Rational Rational::operator--(int x)
{
    Rational tmp =*this;
    num-=den;
    return tmp;
}
ostream & operator<<(ostream &out,const Rational &r)
{
    out<<r.num<<"/"<<r.den;
    return out;
}

```

```

istream &operator>>(istream &in,Rational &r)
{
    in>>r.num>>r.den;
    r.ReductFraction();
    return in;
}

int main()
{
    Rational r1,r2,r3,r4;
    double x;
    cout<<"请输入两个分数: "<<endl;
    cin>>r1>>r2;
    r3=r1+r2;
    cout<<r1<<"+"<<r2<<"="<<r3<<endl;
    r3=r1*r2;
    cout<<r1<<"*"<<r2<<"="<<r3<<endl;
    r4=(r1+r2)*r3;
    cout<<"("<<r1<<"+"<<r2<<")*"<<r3<<"="<<r4<<endl;
    cout<<(r1<r2?r1:r2)<<endl;
    x=double(r1);
    cout<<x<<endl;
}

```

DoubleArray

```

#ifndef 课后习题_DOUBLE_ARRAY_H
#define 课后习题_DOUBLE_ARRAY_H

#include <iostream>
using namespace std;
class DoubleArray
{
    friend ostream& operator<<(ostream& out,const DoubleArray& arr)
    {
        out<<"数组内容: \n";
        for(int i=arr.low;i<=arr.high;i++)
            out<<arr[i]<<"\t";
        out<<endl;
        return out;
    }
    friend istream& operator>>(istream& in,DoubleArray& arr)
    {
        cout<<"输入数组内容: \n";
    }
}

```

```

        for(int i=arr.low;i<=arr.high;i++)
            in>>arr[i];
        return in;
    }
    friend bool operator==(const DoubleArray& arr1,const DoubleArray&
arr2)
    {
        if(arr1.low!=arr2.low||arr1.high!=arr2.high) return false;
        for(int i=arr1.low;i<=arr1.high;i++)
            if(arr1[i]!=arr2[i]) return false;
        return true;
    }
private:
    int low;
    int high;
    double *storage;
public:
    DoubleArray(int lh=0,int rh=0);
    DoubleArray(const DoubleArray &arr);
    DoubleArray(DoubleArray &&other);
    DoubleArray& operator=(const DoubleArray &right); //拷贝赋值
    DoubleArray& operator=(DoubleArray &&right);      //移动赋值
    double &operator[](int index);                    //作为左值
    const double &operator[](int index) const;        //作为右值
    DoubleArray operator()(int start,int end,int lh); //截取子数组
    ~DoubleArray();

};
#endif //课后习题_DOUBLE_ARRAY_H

```

```

#include "Double_Array.h"
#include <iostream>
#include <cassert> // assert 宏
using namespace std;
DoubleArray::DoubleArray(int lh,int rh):low(lh),high(rh)
{
    if(lh>rh)
    {
        cout<<"error:low>high"<<endl;
        exit(1);
    }
    storage=new double[high-low+1];
    for(int i=0;i<=high-low;++i)
        storage[i]=0;
}

```



```

}
DoubleArray::DoubleArray(const DoubleArray &right)
{
    low=right.low;
    high=right.high;
    storage=new double[high-low+1];
    for(int i=0;i<=high-low;++i)
        storage[i]=right.storage[i];
}
DoubleArray::DoubleArray(DoubleArray &&other) : low(other.low),
high(other.high), storage(other.storage)
{
    other.storage = nullptr;
}
DoubleArray &DoubleArray::operator=(const DoubleArray &right)
{
    if(this==&right) return *this;
    delete [] storage;
    low=right.low;
    high=right.high;
    storage=new double[high-low+1];
    for(int i=0;i<=high-low;++i)
        storage[i]=right.storage[i];
    return *this;
}
DoubleArray &DoubleArray::operator=(DoubleArray &&right)
{
    delete [] storage;
    low = right.low;
    high = right.high;
    storage = right.storage;
    right.storage = nullptr;
    return *this;
}
double &DoubleArray::operator[](int index)
{
    assert(index>=low&&index<=high); // assert 宏:若条件为假, 则输出
    错误信息并终止程序运行
    return storage[index-low];
}
const double &DoubleArray::operator[](int index) const
{
    assert(index>=low&&index<=high);
    return storage[index-low];
}

```

```

}
DoubleArray DoubleArray::operator() (int start, int end, int lh)
{
    assert (start<end&&start>=low&&end<=high);
    DoubleArray tmp(lh, lh+end-start);
    for(int i=0; i<=end-start+1; ++i)
        tmp.storage[i]=storage[i+start-low];
    return tmp;
}
DoubleArray::~~DoubleArray()
{
    delete [] storage;
}
int main()
{
    DoubleArray array1(20, 30), array2;
    cin>>array1;
    cout<<"array1 的内容为:"<<array1<<endl;
    cout<<"array2 的内容为:"<<array2<<endl;
    array2=array1;
    cout<<"array2 的内容为:"<<array2<<endl;
    cout<<((array1==array2)?"array1==array2":"array1!=array2")<<endl;
    array2[25]=0;
    cout<<((array1==array2)?"array1==array2":"array1!=array2")<<endl;
    array2 = array1(22, 25, 2);
    cout<<"array2 的内容为:"<<array2<<endl;
    return 0;
}

```